

Sistema de Gestión de Clientes y Productos

Se trabajó un sistema CRUD que permite controlar clientes y productos mediante TypeScript con Node.js para su ejecución, se trabajo con interfaces, type y enums para darle una estructura más profesional y aplicando buenas practicas dentro del proyecto.

1. Diseño de los Modelos

Cliente

Representa a los usuarios que realizan transacciones en el sistema. Su diseño abarca propiedades de identificación legal, información de contacto y control de fidelidad:

- **id (number):** Identificador numérico único y obligatorio para el control lógico e indexación dentro del sistema.
- **nombre (string):** Nombre completo del cliente para personalización y facturación.
- **edad (number):** Edad del usuario, indispensable para reglas comerciales específicas.
- **dpi (number):** Documento Personal de Identificación, modelado de forma numérica para el registro e identificación legal.
- **correo (string) y telefono (number):** Canales primarios de contacto para notificaciones y envío de comprobantes.
- **estado (Estado):** Atributo basado en un tipo específico que define si el cliente está hábil para operar.
- **tipo_cliente (Tipo):** Clasificación que segmenta al usuario según su nivel de interacción con la tienda.

Producto

Define las propiedades de los artículos disponibles en el catálogo de ventas. El modelo incluye control financiero, logístico y temporal:

- **id (number):** Clave numérica única de identificación para la gestión del inventario.
- **nombre (string):** Nombre comercial identificativo del artículo.
- **precio (number):** Valor monetario asignado para la venta al público.
- **stock (number):** Existencias o unidades físicas disponibles en el almacén.
- **categoria (Categoria):** Clasificación que encasilla el producto dentro de un rubro específico.
- **estado (Estado):** Define la disponibilidad de exhibición o discontinuación del artículo.

- **fecha_registro (string):** Cadena que guarda el momento exacto en el que el producto ingresó al inventario.

2. Enumeraciones y Tipos Creados

Para mitigar errores humanos y restringir los valores permitidos a opciones de negocio válidas, se implementaron combinaciones de estructuras:

- **Enumeración Tipo:** Un enum basado en cadenas de texto que define de forma estricta los niveles de lealtad de un cliente:
 1. Inicial
 2. Frecuente
 3. Fiel
- **Unión de Tipos Categoría:** Un tipo literal cerrado que restringe las categorías posibles del catálogo exclusivamente a: "hogar", "comida", "electronico" u "otro".
- **Unión de Tipos Estado:** Un tipo literal estandarizado de uso compartido para controlar la persistencia o visibilidad lógica de las entidades: "activo" o "inactivo".

3. Reglas de Validación y Transformación Aplicadas

Conversión Obligatoria de Tipos (Parsing)

- **Captura:** Toda la información que ingresa desde la terminal se recibe inicialmente en formato de texto plano (string).
- **Acción:** Antes de procesar cualquier operación, los datos numéricos esenciales (como identificadores, precios, existencias o documentos de identidad) se transforman forzosamente a tipos numéricos puros. Esto evita incompatibilidades y asegura la integridad matemática del sistema.

Normalización por Selección Cerrada

- **Acción:** Para simplificar la experiencia del usuario y evitar errores de escritura, la interfaz ofrece menús con opciones numéricas predefinidas (por ejemplo, seleccionar del 1 al 3 para clasificaciones o estados).
- **Control:** El sistema intercepta este número y lo traduce internamente a los valores o etiquetas de negocio reales. Si el usuario ingresa un valor fuera del rango permitido, un mecanismo de protección asigna automáticamente una categoría o estado por defecto (como "otro" o "activo"), impidiendo que el programa falle.

Protección ante Registros Inexistentes

- **Acción:** Antes de realizar cualquier modificación o eliminación, el sistema realiza una búsqueda obligatoria para verificar si el identificador solicitado realmente existe en el almacenamiento.
- **Control:** Si el registro no es localizado, la operación se detiene de inmediato de forma segura y se emite un mensaje preventivo en la pantalla, bloqueando cualquier intento de alteración errónea en los datos.

4. Persistencia de Datos mediante Archivos JSON

Como mejora a la arquitectura inicial del sistema, se incorporó un mecanismo de persistencia basado en archivos JSON, permitiendo conservar la información de clientes y productos incluso después de finalizar la ejecución del programa.

Esta estrategia elimina la dependencia exclusiva de estructuras almacenadas en memoria y facilita la simulación del comportamiento de una base de datos ligera para aplicaciones de consola.

La información se organiza en dos archivos independientes:

- **Clientes.json:** almacena todos los registros correspondientes a clientes.
- **Productos.json:** almacena todos los registros relacionados con el inventario de productos.

Cada archivo contiene un arreglo de objetos serializados en formato JSON, permitiendo una lectura y escritura sencilla utilizando las herramientas proporcionadas por Node.js.

5. Módulo de Persistencia

Con el objetivo de mantener una arquitectura modular y cumplir el principio de responsabilidad única, se implementó un módulo dedicado exclusivamente a la lectura y escritura de información.

Lectura de información

El archivo **Read.ts** contiene funciones especializadas para recuperar los datos almacenados.

Entre sus responsabilidades destacan:

- Abrir el archivo JSON correspondiente.
- Leer su contenido como texto.

- Convertir la información mediante **JSON.parse()**.
- Retornar arreglos tipados de clientes o productos.
- Capturar excepciones en caso de que el archivo no exista o no pueda leerse.

En caso de error, el sistema devuelve un arreglo vacío, permitiendo que la aplicación continúe funcionando sin finalizar inesperadamente.

Escritura de información

El archivo **Write.ts** centraliza toda la persistencia del sistema.

Cada vez que ocurre una operación CRUD, las modificaciones son serializadas mediante **JSON.stringify()** y almacenadas nuevamente dentro del archivo correspondiente.

Para mejorar la legibilidad de los archivos, la serialización utiliza formato indentado, facilitando la inspección manual de la información.

6. Arquitectura del Proyecto

La nueva organización del proyecto separa claramente las responsabilidades de cada componente.

Modelos

Representan la estructura de las entidades Cliente y Producto mediante interfaces de TypeScript.

Enumeraciones y Tipos

Controlan los valores válidos que puede recibir cada atributo del sistema.

Servicios

Los módulos **ClienteService** y **ProductoService** contienen toda la lógica de negocio del CRUD.

Entre sus responsabilidades se encuentran:

- Agregar registros.
- Buscar información.
- Actualizar datos.

- Eliminar registros.
- Validar reglas del negocio.
- Solicitar la persistencia al módulo correspondiente.

Persistencia

Los módulos Read y Write administran exclusivamente el acceso a los archivos JSON, desacoplando completamente la lógica del almacenamiento respecto a la lógica del negocio.

7. Nuevas Validaciones Implementadas

Además de las validaciones descritas anteriormente, el sistema incorpora nuevas reglas para garantizar la consistencia de la información.

Validación de Identificadores Duplicados

Antes de registrar un cliente o producto, el sistema verifica que no exista previamente otro registro con el mismo identificador.

Si el ID ya existe, la operación se cancela y se informa al usuario.

Validación del Porcentaje de Descuento

El modelo Producto incorpora un nuevo atributo denominado **descuento**.

Para asegurar valores válidos, únicamente se aceptan porcentajes comprendidos entre:

- 0%
- 100%

Si el usuario intenta ingresar un valor fuera de ese rango, la operación es cancelada inmediatamente.

Registro Automático de Fecha

Cuando un nuevo producto es agregado al inventario, el sistema registra automáticamente la fecha en que fue creado.

Este valor se almacena dentro del atributo:

- **fecha_registro**

evitando que el usuario tenga que ingresarlo manualmente.

8. Cálculo Automático de IVA

Como funcionalidad adicional, el sistema incorpora una operación para calcular automáticamente el costo final de un producto.

El proceso consiste en:

- Obtener el precio base del producto.
- Aplicar el IVA del 12%.
- Aplicar posteriormente el descuento registrado.
- Mostrar un resumen completo de la operación.

La información presentada incluye:

- Identificador del producto.
- Nombre.
- Subtotal.
- IVA aplicado.
- Descuento aplicado.
- Total final.

Esta funcionalidad facilita la simulación del precio final de venta sin modificar los datos almacenados.

9. Manejo de Errores

Para incrementar la robustez del sistema, todas las operaciones de lectura y escritura utilizan bloques **try-catch**.

Este mecanismo permite controlar errores como:

- Archivos inexistentes.
- Problemas de lectura.
- Problemas de escritura.
- Información JSON inválida.

En lugar de finalizar abruptamente la ejecución, el sistema informa el problema mediante mensajes descriptivos y continúa funcionando siempre que sea posible.